ENSF 612 Project Proposal (Milestone 1)

Engineering Large-Scale Data Analytics Systems, Fall 2026
Instructor: Dr. Ebrahim Karami
October 21, 2026

1 Project Title

DupHunter: Finding Duplicate Questions on Stack Overflow and Measuring the Impact of AI

2 Group Members

Name	Main role
Satyam Bhanot	Data pipeline, Spark cluster setup, speed and scaling tests
Iesrat Jahan	Duplicate detection (shingling, MinHash, LSH)
Md Jubayer Khan	Popular tag combinations and expert ranking (SON, PageRank)
Navjot Singh Shienh	Stream counting and the AI-impact analysis

3 Problem Statement

Stack Overflow is a website where programmers ask and answer questions. Many new questions have already been asked before, just in different words. These are called *duplicates*. Today, volunteers find and close them by hand, and many are missed.

We want to build a system that finds duplicate questions automatically. The hard part is size. Stack Overflow has about 24 million questions. Comparing every question with every other question would take years on one computer, so we need methods that are built for large data and can run on many computers at once.

We will also use the same data to look at a second question: how have AI tools such as ChatGPT and GitHub Copilot changed Stack Overflow since they became popular in 2022?

Questions we want to answer:

Q1. Can we find most of the known duplicate questions without comparing every pair?
Q2. How much faster does our system run when we add more computers?
Q3. Which tags often appear together, and who are the top experts for each tag?
Q4. How did the number of new questions and duplicates change after AI tools became popular, and which topics changed the most?

Expected outcome: a working Spark pipeline that answers these four questions, with measurements of both accuracy and speed.

4 Motivation and Application

Duplicate questions waste time. People wait for answers that already exist, experts answer the same thing again, and volunteers spend hours searching. A tool that finds duplicates could show a user an existing answer before they even post. The same idea is used to remove repeated pages from web search results and from AI training data.

The AI question is timely. A 2024 study found that Stack Overflow activity dropped after ChatGPT was released [3]. We want to check this across all questions, see which topics were hit hardest, and see whether duplicates changed too.

This project fits ENSF 612 because the data is too large for simple methods. It needs the course's tools for large data: MapReduce and Spark, similarity search, frequent itemsets, stream algorithms, and PageRank [1].

5 Dataset

Source. We will use the public Stack Exchange data dump on the Internet Archive (`https://archive.org/details/stackexchange`). We checked it on October 21. The newest release is from September 30, 2026 [2]. It is free to use under the CC BY-SA 4.0 license.

Size and format. The Stack Overflow archive is a single 64.2 GiB (68.9 GB) compressed file (`.7z`)

covering July 2008 to September 2026. Inside, each table is an XML file with one record per line. The dump has many tables, but we only need the three in Table 1. Record counts are estimates from older releases; we will update them after loading the data.

Table 1: The data we will use.

Table	Records (approx.)	Fields we keep	What we use it for
Posts	60 million (24 million questions)	id, type, parent id, title, body, tags, date, author	Dates and tags for all questions; full text for a subset
PostLinks	10 million	post id, related post id, link type	Known duplicate pairs (our answer key)
Users	22 million	id, reputation	Checking our expert ranking
Super User site	1.4 GB compressed	same tables	Testing our code

How much data we will use. Each question needs a different amount of data, so we will only load what each part needs.

- We keep only the columns listed above and skip every other table.
- Dates and tags for all 24 million questions are small. They are enough for the tag combinations (Q3) and the AI trend analysis (Q4).
- Duplicate detection (Q1) is the heavy part because it needs the full question text, and we need to run it many times to test different settings. We will run it on questions tagged `python` (roughly 2 million; we will confirm the count), adding more tags if time allows. This is still a large-scale problem: 2 million questions make about 2 trillion pairs. We only score duplicate pairs where both questions are in the subset.
- We will build and debug everything on the Super User site from the same dump. It has the same format but is about 50 times smaller.

The scaling tests (Q2) on growing parts of the subset will show how run time grows, so we can estimate the cost of running on all 24 million questions.

Data quality and limitations.

- Question text is stored as HTML, with code mixed into the writing.
- Deleted posts are missing, and deleted users show up as empty values.
- Not every duplicate was marked by volunteers, so our answer key is incomplete. Some of our matches may be real duplicates that were never marked, so our precision will be a lower bound.
- The most recent months are incomplete, because new questions may still be answered or closed.
- Working on one tag means our duplicate results may not hold for every topic.

Preprocessing. We will:

1. Extract only the three tables we need and read them as a stream, so we never store hundreds of GB of unpacked XML.
2. Keep only the needed columns and save them in Parquet format (a compressed table format that Spark reads quickly), split by year.
3. For the subset, remove HTML tags and keep code blocks separate from the text.
4. Break each subset question into short groups of words, called *shingles*, for duplicate detection.

6 Proposed Methodology

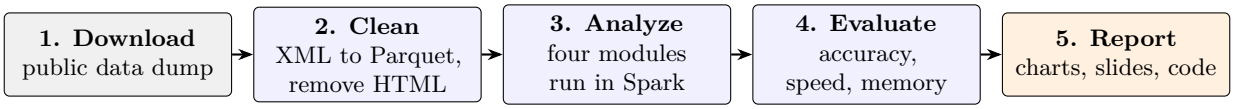


Figure 1: Overview of the DupHunter pipeline.

Figure 1 shows the main steps. After cleaning, the data is shared by four modules, one per team member:

- **M1, Duplicate detection (Q1).** Turn each question into shingles, compress the shingles into a short MinHash *signature*, then use LSH to put similar questions in the same bucket. We only

compare questions that share a bucket, which avoids comparing every pair.

- **M2, Tags and experts (Q3).** Treat each question's tags as a "basket" and find tag combinations that often appear together. Build a graph of who answers whose questions and run PageRank for each tag to find the top experts.
- **M3, Stream counting (Q4).** Replay the posts in date order, as if they were arriving live, and use small-memory counters to track trending tags, active users, and duplicate rates over time.
- **M4, AI impact and system tests (Q2, Q4).** Compare monthly trends before and after AI tools became popular (June 2022 for Copilot, November 2022 for ChatGPT). Run speed and scaling tests on all modules.

Running on many computers. We will use Apache Spark (in Python) [5], which follows the MapReduce model from the course [4]. Spark splits the data into chunks and processes them on several computers at the same time. To avoid running out of memory, we will process the LSH buckets in smaller batches and keep large intermediate results on disk.

How we will evaluate.

- **Accuracy (Q1):** Recall@K and Precision@K (how many known duplicates we find, and how many of our top K matches are correct), MAP (how high real duplicates rank), and reduction ratio (the share of pairs we never compare). We also check against exact results on a sample.
- **Correctness (Q3, Q4):** compare our own code to Spark's built-in versions and to exact counts.
- **Speed (Q2):** run time, memory use, and *speedup*, meaning how many times faster the job runs on more computers. We will test 1, 2, 4, and 8 workers and 10%, 25%, 50%, and 100% of the subset.

7 Algorithms and Techniques

We will write the main course algorithms ourselves and compare them with ready-made Spark tools, so the report can clearly show which code is ours.

Table 2: Algorithms we will build ourselves and what we compare them to.

Module	Course algorithms (our own code)	Compared with	Why we need it
M1	Shingling, MinHash, LSH [6, 7]	Spark `MinHashLSH`; exact check on a sample	Find similar questions without comparing all pairs
M2	SON frequent itemsets [8]; topic-sensitive PageRank [9]	Spark `FPGrowth`; GraphFrames PageRank	Popular tag combinations; top experts per tag
M3	Count-Min Sketch [10], DGIM [11], Flajolet-Martin [12]	Exact counts in Spark SQL	Count trends with very little memory
M4	Before/after trend comparison; MapReduce-style Spark jobs	Same jobs on 1 vs. many workers	Measure AI impact and system speedup

8 Expected Challenges

Challenge	How we plan to handle it
Duplicates written in very different words	Measure how often this happens; try a second similarity method if time allows
Missing duplicate labels	Treat precision as a lower bound; rely mainly on recall, which the missing labels do not affect
Very large file and limited memory	Read only the needed tables and columns; use a subset; test on Super User first
Some LSH buckets become huge (common phrases)	Limit bucket size and split large buckets
Choosing LSH settings (number of bands and rows)	Too few bands miss duplicates and too many give false matches; test several settings on Super User
Moving a lot of data between computers when grouping LSH buckets	Process the buckets in batches; measure data moved and time per step
PageRank on a graph with millions of users is slow	Keep the graph split the same way each round so little data moves; stop when the ranks stop changing

9 Expected Results

We expect that:

- LSH will find most known duplicates while skipping more than 99.9% of all question pairs.
- Simple steps will speed up almost in line with the number of workers, while steps that move a lot of data between computers will speed up less.
- The small-memory counters will stay close to the exact counts while using far less memory.
- Common tag combinations will match real technology groups (for example, `python`, `pandas`, and `dataframe`), and our top experts will mostly be high-reputation users.
- New questions will drop after AI tools became popular, more in some topics than others.

10 Initial Project Schedule

The cleaned Super User data will be ready by October 24, so all four modules can start in parallel; Stack Overflow (SO) runs follow.

Table 3: Weekly plan (blue: main work; orange: milestone).

Task (week starting)	Who	Oct 21	Oct 28	Nov 4	Nov 11	Nov 18	Nov 25	Dec 2
Clean data (Super User, then SO)	Satyam	Oct 24						
Cluster setup and speed tests	Satyam							
Duplicate detection (M1)	Iesrat							
Tags and experts (M2)	Jubayer							
Streams and AI impact (M3, M4)	Navjot							
Full runs and results	All					Nov 21		
Presentation	All							
Final report and code	All							Dec 4

11 References

[1] A. Rajaraman and J. D. Ullman, *Mining of Massive Datasets*. Cambridge University Press, 2011.

[2] Stack Exchange, Inc., "Stack Exchange Data Dump 2026-09-30," Internet Archive, https://archive.org/details/stackexchange_20260930.

[3] R. M. del Rio-Chanona, N. Laurentsyeva, and J. Wachs, "Large language models reduce public knowledge sharing on online Q&A platforms," *PNAS Nexus*, vol. 3, no. 9, 2024.

[4] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," in *Proc. OSDI*, 2004.

[5] M. Zaharia *et al.*, "Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing," in *Proc. NSDI*, 2012.

[6] A. Z. Broder, "On the resemblance and containment of documents," in *Proc. Compression and Complexity of Sequences*, 1997.

[7] P. Indyk and R. Motwani, "Approximate nearest neighbors: Towards removing the curse of dimensionality," in *Proc. ACM STOC*, 1998.

[8] A. Savasere, E. Omiecinski, and S. Navathe, "An efficient algorithm for mining association rules in large databases," in *Proc. VLDB*, 1995.

[9] T. H. Haveliwala, "Topic-sensitive PageRank," in *Proc. WWW*, 2002.

[10] G. Cormode and S. Muthukrishnan, "An improved data stream summary: The count-min sketch and its applications," *J. Algorithms*, vol. 55, no. 1, 2005.

[11] M. Datar, A. Gionis, P. Indyk, and R. Motwani, "Maintaining stream statistics over sliding windows," *SIAM J. Comput.*, vol. 31, no. 6, 2002.

[12] P. Flajolet and G. N. Martin, "Probabilistic counting algorithms for data base applications," *J. Comput. Syst. Sci.*, vol. 31, no. 2, 1985.